Focal X Company

12/1/2026, 06:00 PM
25/1/2026, 06:00 PM

OWASP Juice Shop Penetration Testing Report

By : Judy Muhammad Darwish

Table of Contents

1. Introduction
2. Executive Summary
3. Scope
4. Testing Environment
5. Methodology
6. Web Reconnaissance
7. Vulnerability Discovery
8. Vulnerability Summary Table
9. Severity & Impact Assessment
10. Tools Used
11. Conclusion
12. References

Introduction

This project presents a comprehensive penetration test of the OWASP Juice Shop application. The objective of the assessment was to identify security vulnerabilities and evaluate the application's resilience against common cyberattacks.

The test was conducted in a controlled and isolated environment using Docker on Kali Linux, following globally recognized OWASP methodologies. This report documents the discovered vulnerabilities,

analyzes their impact, and provides actionable recommendations for remediation.

Executive Summary

A full security assessment was performed on the OWASP Juice Shop application using OWASP WSTG and PTES methodologies. The evaluation covered the frontend, backend APIs, authentication mechanisms, and authorization logic.

Multiple vulnerabilities were identified across the OWASP Top 10 categories, including:

- Authentication weaknesses
- Broken access control
- SQL injection
- Cryptographic failures
- Security misconfigurations
- Insecure design

These findings indicate insufficient security controls, lack of proper authorization, weak input validation, and inadequate protection of sensitive data.

Overall security posture: Requires significant improvement.

Scope

Target Application: OWASP Juice Shop (Docker version)

Included Components:

- Frontend

- Backend APIs
- Authentication & Authorization
- Input and search forms

Targeted Vulnerabilities: OWASP Top 10

Testing Environment

- **Operating System:** Kali Linux 2024
- **Platform:** Docker
- **Application URL:** http://localhost:8080
- **Tools Used:** Burp Suite, Browser DevTools, JavaScript Beautifier
- **Connection Method:** HTTP (Localhost)

Methodology

The penetration test followed international standards:

- **OWASP Web Security Testing Guide (WSTG)**
- **PTES (Penetration Testing Execution Standard)**

Testing Phases

1. Reconnaissance

- Application structure analysis
- Endpoint discovery

2. Scanning & Discovery

- Identifying exposed services

- Mapping attack surface

3. Vulnerability Analysis

- Testing OWASP Top 10 categories
- Evaluating authentication & authorization

4. Exploitation

- Validating exploitability
- Attempting to bypass security controls

5. Reporting & Recommendations

- Documenting findings
- Assessing risks
- Providing remediation steps

Web Reconnaissance

Application Launch

The application was executed using Docker and accessed via:

http://localhost:8080

Application Structure Analysis

Using browser DevTools:

- Observed network traffic
- Monitored API calls
- Tested user registration and search features

This phase helped identify exposed endpoints and backend behavior.

Vulnerability Discovery

Below is a professional English translation of all vulnerabilities identified during testing.

Weak Default Credentials

A weak default admin password (admin123) was discovered, allowing attackers to gain unauthorized administrative access.

Risks

- Full system compromise
- Unauthorized access to sensitive data
- Easy brute-force attacks

Recommendations

- Change all default passwords
- Enforce strong password policies
- Enable MFA

Security Misconfiguration

A hidden route /score-board was discovered inside main.js and accessed manually despite not being visible in the UI.

Risks

- Exposure of internal functionality
- Unauthorized access to sensitive pages

Recommendations

- Enforce server-side authorization

- Avoid exposing sensitive routes in JavaScript
- Minify and optimize JS files

Broken Access Control – Forced Browsing / IDOR in Routes

Admin pages were accessible by manually navigating to common admin paths such as:

```
/admin
/administration
/dashboard
/control-panel
/manage
```

Risks

- Unauthorized admin access
- Ability to modify system settings
- Full bypass of access control

Recommendations

- Implement RBAC
- Enforce server-side authorization
- Block direct access to sensitive routes

Privilege Escalation via Role Parameter Tampering

By modifying the login request to include:

```
"role": "admin"
```

the server granted admin privileges to a normal user.

Risks

- Full administrative takeover
- Data manipulation
- Account compromise

Recommendations

- Never accept role values from the client
- Generate roles only from the database
- Validate permissions on every request

SQL Injection – Authentication Bypass

SQL payloads injected into login fields bypassed authentication due to lack of input validation.

Risks

- Login bypass
- Database manipulation
- Full system compromise

Recommendations

- Use parameterized queries
- Validate all inputs
- Hide database error messages

CAPTCHA Bypass

CAPTCHA validation occurred only once. Modified requests were accepted

without re-validation.

Risks

- Replay attacks
- Data manipulation
- Automated abuse

Recommendations

- Validate CAPTCHA on backend
- Invalidate tokens after use
- Block repeated modified requests

Authentication Failures (No Rate Limiting)

Unlimited login attempts were allowed without delays or CAPTCHA.

Risks

- Brute-force attacks
- Credential stuffing

Recommendations

- Implement rate limiting
- Add exponential backoff
- Trigger CAPTCHA after repeated failures

Broken Access Control – IDOR in Basket API

Changing the basketId returned another user’s basket.

Risks

- Access to other users’ data
- Manipulation of cart items

Recommendations

- Validate ownership on server
- Do not trust client-side identifiers

Cross-Site Scripting (XSS)

JavaScript payloads executed inside the browser due to lack of input sanitization.

Risks

- Token theft
- Phishing
- Unauthorized actions

Recommendations

- Sanitize inputs
- Encode outputs
- Implement CSP

Cryptographic Failures

Password change requests transmitted:

- Old password
- New password
- Hash

- User data

in plaintext.

Risks

- Credential exposure
- Account takeover

Recommendations

- Use HTTPS/TLS
- Apply strong hashing (bcrypt, Argon2)
- Avoid sending sensitive data in plaintext

Insecure Design

The API accepted manipulated quantity values without proper backend validation.

Risks

- Business logic abuse
- Data integrity issues

Recommendations

- Enforce strict backend validation
- Reject abnormal values

Vulnerability Summary Table

Vulnerability	Category	Severity	Impact
Weak Default Credentials	Authentication	High	Admin access
Security Misconfiguration	Configuration	Medium	Hidden route exposure
Forced Browsing / IDOR	Access Control	High	Unauthorized admin access
Role Tampering	Privilege Escalation	Critical	Full admin takeover
SQL Injection	Injection	Critical	Login bypass
CAPTCHA Bypass	Authentication	Medium	Replay attacks
No Rate Limiting	Authentication	High	Brute-force
Basket IDOR	Access Control	High	Data exposure
XSS	Injection	High	Token theft
Cryptographic Failures	Cryptography	High	Password exposure
Insecure Design	Design	Medium	Logic manipulation

Severity & Impact Assessment

Critical: SQL Injection, Role Tampering **High:** XSS, IDOR, Weak Credentials, Cryptographic Failures **Medium:** CAPTCHA Bypass, Misconfiguration, Insecure Design

Tools Used

- Kali Linux 2024
- Docker
- Burp Suite
- Browser DevTools
- JavaScript Beautifier
- CrackStation

Conclusion

The assessment revealed multiple vulnerabilities across authentication, authorization, input validation, and cryptographic controls. These weaknesses could lead to full system compromise, data leakage, and unauthorized access

References:

OWASP Top 10 – 2021
OWASP Web Security Testing Guide (WSTG)
OWASP Juice Shop Documentation
Penetration Testing Execution Standard (PTES)
Burp Suite Documentation – PortSwigger
Docker Documentation
CrackStation – Password Hash Cracking
wrk – HTTP Benchmarking Tool
Mozilla Developer Tools (DevTools) Documentation